

Software Design Document (SDD)

Module: uart_driver - USART2 Transmission Driver

Version: 1.0

Authors: Steven McClellan, Vania Leal, Leonardo García, Melanie Picen

Item	Description
Source file	uart_driver.c
Main purpose	Initialize USART2 and transmit one byte through polling-based UART transmission.
Main public functions	uart_init(), uart_write()
Target peripheral	USART2 on STM32-style register map

1. Introduction

1.1 Purpose

This document describes the design and architecture of the uart_driver module. The module provides a compact UART transmit interface for USART2. It enables the USART2 peripheral clock, configures the baud-rate register, enables the transmitter and peripheral, and provides a blocking byte-write function based on TXE status polling.

1.2 Scope

The uart_driver module is a low-level embedded driver. It directly accesses RCC and USART2 registers through symbols and macros provided by uart_driver.h. The current implementation supports transmit-only operation: uart_init() prepares USART2 and uart_write() sends a single byte. GPIO alternate-function configuration is assumed to be performed elsewhere before using the UART output pin.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
UART	Universal Asynchronous Receiver/Transmitter serial communication interface.
USART2	STM32 USART peripheral used by this driver.
RCC	Reset and Clock Control block. The driver enables the USART2 clock through RCC_APB1ENR.
BRR	Baud Rate Register. Holds UART_BRR_VALUE to configure the communication speed.
CR1	USART Control Register 1. Used to enable the transmitter and the USART peripheral.
SR	USART Status Register. Contains the TXE flag used before writing data.
DR	USART Data Register. Writing a byte to DR starts transmission.
TXE	Transmit Data Register Empty flag. When set, DR is ready for a new byte.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	uart_init()	The system shall enable the USART2 peripheral clock through RCC_APB1ENR.

Req. ID	Function	Requirement Statement
FR-2	uart_init()	The system shall configure the USART2 baud rate using UART_BRR_VALUE.
FR-3	uart_init()	The system shall enable the USART2 transmitter and peripheral using CR1.TE and CR1.UE.
FR-4	uart_write()	The system shall wait until TXE is set before writing a byte to USART2->DR.
FR-5	uart_write()	The system shall return UART_ERR_TIMEOUT if TXE does not become ready before the timeout counter expires.

Non-functional requirements: no dynamic memory allocation; minimal register-level implementation; blocking and polling-based transmission; timeout protection in uart_write(); no HAL middleware dependency inside this module.

2. System Architecture

2.1 High-Level Design

The UART driver sits between the application layer and the USART2 hardware registers. It exposes a small public API and hides the register operations required for basic transmit operation.

Layer	Files / Components	Description
Application Layer	main.c or higher-level module	Calls uart_init() once and calls uart_write() for byte transmission.
UART Driver Layer	uart_driver.c / uart_driver.h	Implements USART2 initialization and blocking byte transmission.
Register Interface	RCC_APB1ENR, USART2->BRR, CR1, SR, DR	Provides direct access to RCC and USART2 peripheral registers.
Hardware Layer	STM32 USART2 peripheral and UART TX pin	Physically shifts the transmitted byte on the configured UART TX line.

2.2 Class Diagram

Class Diagram - uart_driver module

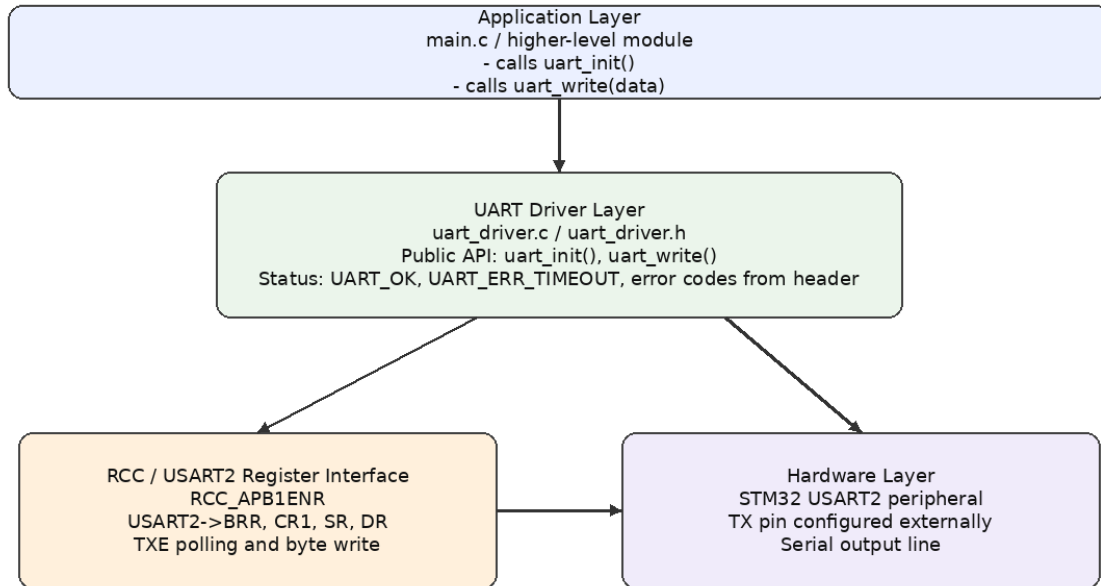


Figure 1. Class Diagram - UART driver layered structure.

Figure 1. Class Diagram - UART driver layered structure.

2.3 Module Interfaces

Interface	Description
Public API	<code>uart_init()</code> and <code>uart_write()</code> declared in <code>uart_driver.h</code> .
External macros and registers	<code>RCC_APB1ENR</code> , <code>RCC_APB1ENR_USART2EN_POS</code> , <code>USART2</code> , <code>UART_BRR_VALUE</code> , <code>USART_CR1_TE_POS</code> , <code>USART_CR1_UE_POS</code> , <code>USART_SR_TXE_POS</code> .
Return type	<code>uart_status_t</code> , with <code>UART_OK</code> and <code>UART_ERR_TIMEOUT</code> used by the implementation.
Assumptions	GPIO TX pin alternate-function setup is performed outside this module. The implementation is transmit-only and does not enable receive mode.

3. Detailed Design

3.1 Module Behavior

Sequence Diagram - uart_init()

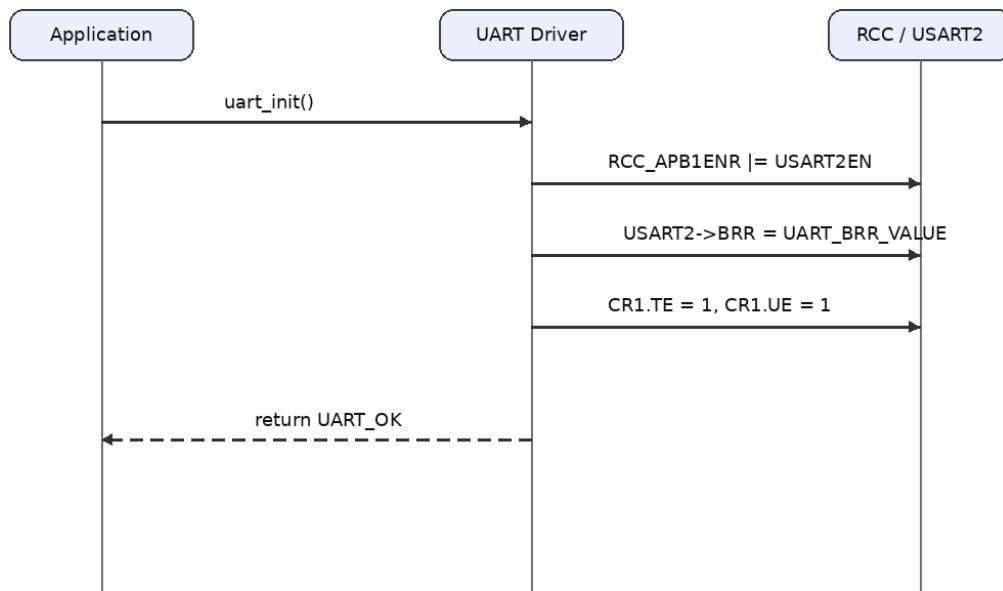


Figure 2. Sequence Diagram - uart_init.

Figure 2. Sequence Diagram - uart_init.

Activity Diagram - uart_init()

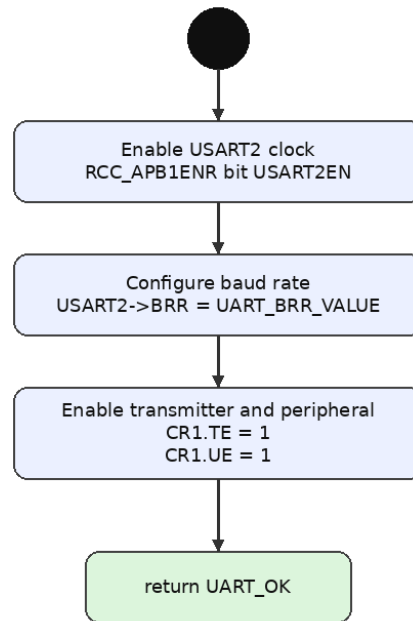


Figure 3. Activity Diagram - uart_init.

Figure 3. Activity Diagram - uart_init.

Sequence Diagram - uart_write(data)

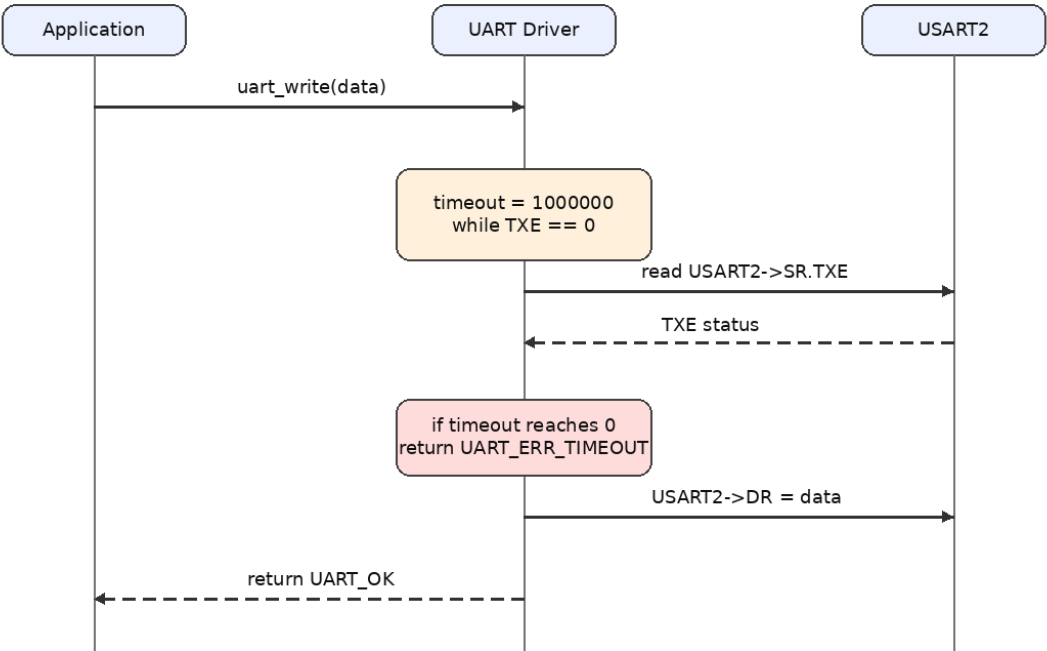


Figure 4. Sequence Diagram - uart_write.

Figure 4. Sequence Diagram - uart_write.

Activity Diagram - uart_write(data)

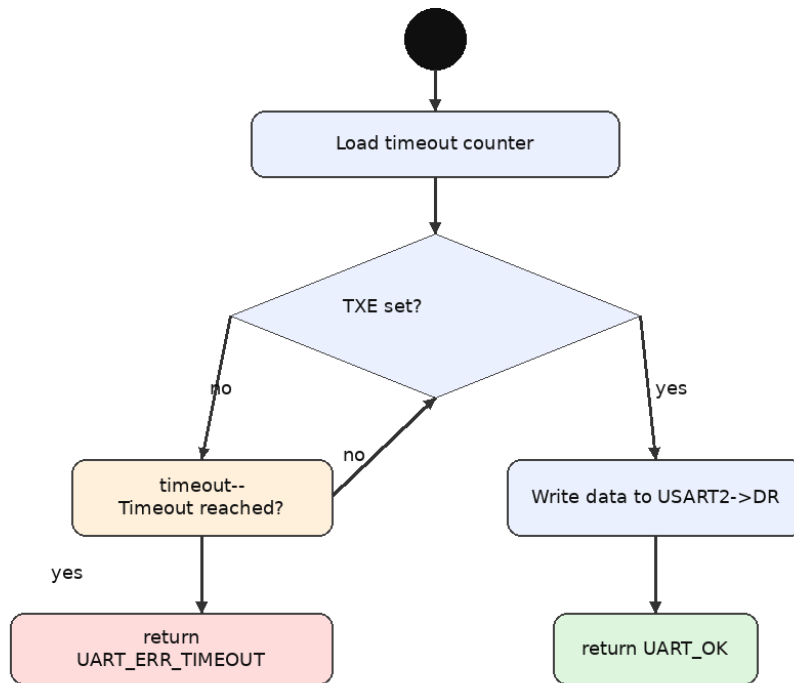


Figure 5. Activity Diagram - uart_write.

Figure 5. Activity Diagram - uart_write.

State Diagram - UART driver lifecycle

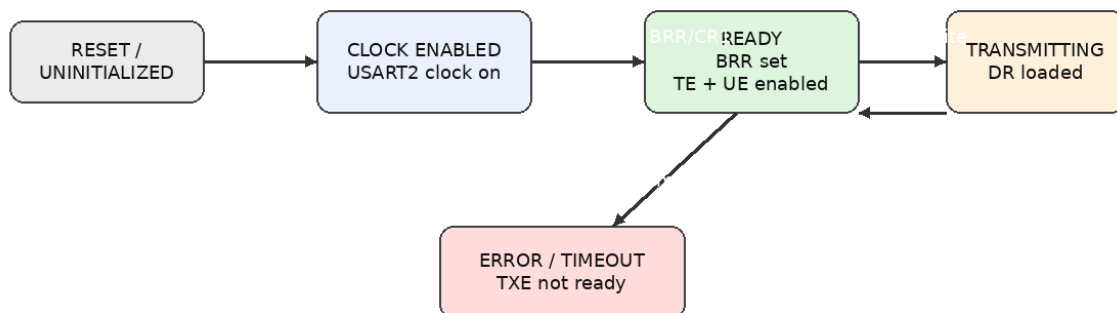


Figure 6. State Diagram - UART driver lifecycle.

Figure 6. State Diagram - UART driver lifecycle.

3.2 uart_driver Module

The driver consists of two public operations. Initialization is performed by `uart_init()`, which enables the peripheral clock, writes the baud-rate value, and enables the USART2 transmitter and peripheral. Transmission is performed by `uart_write()`, which waits until the TXE flag indicates that USART2->DR is available, then writes the byte into the data register.

3.3 Initialization Flow

`uart_init()` enables the USART2 clock by setting the corresponding bit in `RCC_APB1ENR`. It then writes `UART_BRR_VALUE` into `USART2->BRR`. Finally, it enables `CR1.TE` and `CR1.UE` to activate the transmitter and USART2 peripheral. On completion, it returns `UART_OK`.

3.4 Byte Transmission Flow

`uart_write()` uses a polling loop around the TXE flag in `USART2->SR`. A local timeout counter prevents an infinite wait if the transmitter never becomes ready. If TXE becomes set before timeout, the input byte is written to `USART2->DR` and the function returns `UART_OK`. If the timeout expires first, the function returns `UART_ERR_TIMEOUT`.

3.5 Register Usage

Register / Macro	Purpose
<code>RCC_APB1ENR</code>	APB1 peripheral clock enable register. Used to enable USART2 clock.
<code>RCC_APB1ENR_USART2EN_POS</code>	Bit position for USART2 clock enable.
<code>USART2->BRR</code>	Baud-rate configuration register.
<code>UART_BRR_VALUE</code>	Precomputed BRR value used by <code>uart_init()</code> .
<code>USART2->CR1</code>	Control register used to set TE and UE bits.
<code>USART2->SR</code>	Status register polled for TXE.
<code>USART2->DR</code>	Data register where the transmitted byte is written.

4. Application Programming Interfaces

4.1 Data Types and Macros

Type / Macro	Notes / Definition
<code>uart_status_t</code>	Return type used by all public functions. Defined in <code>uart_driver.h</code> .
<code>UART_OK</code>	Return code for successful initialization or transmission.
<code>UART_ERR_TIMEOUT</code>	Return code used when <code>uart_write()</code> cannot observe TXE before timeout expires.
<code>UART_BRR_VALUE</code>	Baud-rate register value loaded into <code>USART2->BRR</code> .
<code>RCC_APB1ENR_USART2EN_POS</code>	Bit position used to enable the USART2 peripheral clock.
<code>USART_CR1_TE_POS</code>	Bit position for transmitter enable.
<code>USART_CR1_UE_POS</code>	Bit position for USART peripheral enable.
<code>USART_SR_TXE_POS</code>	Bit position for the Transmit Data Register Empty flag.

4.2 Error Codes

Code	Notes / Definition
<code>UART_OK</code>	Operation was successful.
<code>UART_ERR_TIMEOUT</code>	<code>uart_write()</code> timeout expired while waiting for <code>USART2->SR.TXE</code> .

4.3 Function Definitions

uart_init

Field	Definition
Function Name	uart_init
Syntax	uart_status_t uart_init(void)
Parameters (in)	None.
Parameters (inout)	None.
Parameters (out)	None.
Return value	UART_OK on successful configuration.
Description	Enables the USART2 clock, loads the baud-rate register, and enables the transmitter and peripheral.
Constraints	Does not configure GPIO alternate function pins. Does not enable receiver mode. Assumes UART_BRR_VALUE and register macros are valid.
Available via	uart_driver.h

uart_write

Field	Definition
Function Name	uart_write
Syntax	uart_status_t uart_write(uint8_t data)
Parameters (in)	data: byte to transmit through USART2.
Parameters (inout)	None.
Parameters (out)	None.
Return value	UART_OK if the byte is written to USART2->DR; UART_ERR_TIMEOUT if TXE does not become ready.
Description	Blocks until USART2 transmit data register is ready, then writes one byte to the data register.
Constraints	Polling-based and blocking. Timeout value is a fixed local constant. It does not verify that uart_init() has already been called.
Available via	uart_driver.h

4.4 Internal Function Definitions

The current implementation has no private helper functions. All logic is contained directly inside `uart_init()` and `uart_write()`.

5. Implementation Notes and Constraints

Topic	Note
Transmit-only behavior	<code>uart_init()</code> enables TE and UE but does not enable RE. The current module is intended for UART transmission only.
GPIO configuration	The driver assumes the USART2 TX pin has already been configured in alternate-function mode by another GPIO module or startup routine.
Blocking transmission	<code>uart_write()</code> blocks while polling TXE. This is simple but can stall the application until TXE is set or timeout expires.
Fixed timeout	The timeout value is hardcoded inside <code>uart_write()</code> . It is not configurable at runtime.
No validation of initialization state	<code>uart_write()</code> does not check whether <code>uart_init()</code> was called previously.

6. Future Enhancements

- Add GPIO alternate-function configuration for USART2 TX inside `uart_init()` or document the external setup requirement in the header.
- Add a string transmission helper such as `uart_write_string()`.
- Enable optional receive support using CR1.RE and a `uart_read()` API.
- Replace polling with interrupt-driven or DMA-based transmission for non-blocking behavior.
- Move the timeout value to a macro or configuration structure.
- Add a status function to check whether TXE is ready without blocking.